

Unidad 4: Replicación y particionamiento

Clase 8 – Escalabilidad, replicación y teorema CAP en NoSQL

Objetivo

Evaluar distintas estrategias de replicación, particionamiento y escalamiento horizontal, aplicando el teorema CAP para tomar decisiones fundamentadas sobre arquitecturas NoSQL.

Tema

Administración y recuperación de datos NoSQL.
Interfaces de administración.
Escalamiento horizontal.
Replicación y particionamiento (profundización).
Teorema CAP aplicado a Mongo, Cassandra y Redis.

Escalabilidad, replicación y teorema CAP en bases de datos NoSQL

Introducción conceptual

Las bases de datos NoSQL surgen como respuesta a problemas que se vuelven difíciles de resolver con arquitecturas centralizadas o con modelos relacionales tradicionales cuando el volumen, la velocidad, la variedad y la distribución de los datos aumentan de manera significativa. NoSQL no significa “ausencia de SQL” en sentido absoluto, sino una familia de modelos de datos no relacionales o no estrictamente relacionales, diseñados para resolver necesidades específicas: documentos flexibles, pares clave-valor de alta velocidad, columnas distribuidas, grafos de relaciones complejas, almacenamiento masivo de eventos o procesamiento de datos semiestructurados.

En este contexto, la escalabilidad no es solo una cuestión de “hacer que el sistema sea más grande”. Escalar una base de datos implica conservar niveles aceptables de rendimiento, disponibilidad, recuperación y confiabilidad a medida que crecen los datos, los usuarios, las consultas, las escrituras y la complejidad operativa. Una arquitectura de datos escalable debe poder absorber más carga sin degradarse de forma crítica y sin depender exclusivamente de un único servidor cada vez más potente.

El problema central aparece cuando una organización necesita procesar millones de eventos, sostener aplicaciones activas las veinticuatro horas, distribuir datos entre regiones, atender picos de demanda, tolerar fallos y recuperar servicios sin pérdida significativa de información. En ese escenario, conceptos como escalamiento horizontal, replicación, particionamiento, consistencia, disponibilidad y tolerancia a

fallos dejan de ser temas técnicos aislados y pasan a formar parte de las decisiones arquitectónicas fundamentales.

Escalabilidad vertical y escalabilidad horizontal

La escalabilidad vertical consiste en aumentar la capacidad de un mismo servidor. Esto puede implicar agregar más memoria RAM, más procesadores, discos más rápidos o mayor capacidad de almacenamiento. Es una estrategia simple de comprender y administrar porque mantiene una arquitectura relativamente centralizada: una base de datos principal, un servidor más potente y una gestión más directa.

El problema de la escalabilidad vertical es que tiene límites físicos, económicos y operativos. Llega un punto en que el servidor no puede seguir creciendo o el costo de ampliarlo se vuelve demasiado alto. Además, si toda la operación depende de una única máquina, se incrementa el riesgo de punto único de falla. Si ese servidor deja de funcionar, el sistema completo puede quedar comprometido.

La escalabilidad horizontal, en cambio, consiste en agregar más nodos al sistema. Un nodo es una instancia, servidor o máquina que participa en el procesamiento o almacenamiento de datos. Cuando varios nodos trabajan coordinadamente, se forma un cluster. El objetivo de un cluster es distribuir carga, datos y procesamiento para que el sistema pueda crecer incorporando nuevos nodos en lugar de depender de un único servidor cada vez más grande.

En bases de datos NoSQL, el escalamiento horizontal es especialmente importante porque muchos de estos motores fueron diseñados para funcionar de manera distribuida. Cassandra, por ejemplo, fue concebida para distribuir datos entre múltiples nodos, replicarlos y permitir altos volúmenes de escritura con una arquitectura descentralizada. MongoDB permite escalar mediante replica sets y sharding. Redis, aunque muchas veces se usa como motor en memoria para operaciones rápidas, también puede operar con replicación, Redis Sentinel o Redis Cluster según los requerimientos de disponibilidad y distribución. La documentación oficial de MongoDB define un replica set como un grupo de procesos mongod que mantienen el mismo conjunto de datos para proveer redundancia y alta disponibilidad; en Cassandra, la documentación destaca que la consistencia puede configurarse por operación mediante niveles de consistencia; y Redis documenta mecanismos de replicación líder-réplica, Sentinel y Cluster para distintos escenarios de operación distribuida.

La diferencia conceptual es importante: escalar verticalmente es “hacer más fuerte una máquina”; escalar horizontalmente es “hacer que varias máquinas trabajen juntas”. Esta segunda opción permite crecer de manera más flexible, pero introduce nuevos problemas: coordinación entre nodos, distribución de datos, sincronización de réplicas, resolución de fallos, latencia de red y consistencia entre copias.

Cluster, nodo y distribución de carga

Un cluster es un conjunto de nodos que actúan de forma coordinada para brindar un servicio de base de datos. Cada nodo puede almacenar una parte de los datos, una

copia de los datos o participar en la coordinación de operaciones. La arquitectura exacta depende del motor utilizado.

En una base distribuida, no alcanza con “tener varios servidores”. La pregunta relevante es qué rol cumple cada servidor. Algunos nodos pueden recibir escrituras, otros pueden responder lecturas, otros pueden actuar como réplicas, otros pueden coordinar operaciones y otros pueden participar en mecanismos de elección o recuperación ante fallos.

La distribución de carga busca evitar que un único nodo concentre todo el trabajo. Si todos los usuarios consultan siempre el mismo nodo, el cluster existe físicamente, pero no está distribuyendo adecuadamente la carga. Si todos los datos más consultados quedan en una sola partición, el diseño genera un cuello de botella. Si las escrituras se concentran siempre en una misma clave, también puede producirse una zona caliente o hot spot.

Un hot spot es una concentración anormal de actividad en una parte específica del sistema. Por ejemplo, si una aplicación de comercio electrónico particiona ventas por país y el noventa por ciento de las operaciones ocurre en Argentina, el nodo o partición asignado a Argentina recibirá mucha más carga que los demás. El sistema estará distribuido, pero mal balanceado.

Por eso, la escalabilidad horizontal requiere pensar cuidadosamente cómo se reparten los datos y cómo se reparten las operaciones. La arquitectura física debe acompañar el patrón real de uso.

Particionamiento y sharding

El particionamiento es la técnica mediante la cual un conjunto de datos se divide en partes más pequeñas para distribuirlas entre distintos nodos. En muchos motores de base de datos distribuidos se utiliza también el término sharding. Un shard es una porción del conjunto total de datos. Cada shard puede residir en uno o varios nodos, según la arquitectura y la configuración de replicación.

La idea básica es simple: en lugar de guardar todos los datos en un único lugar, se dividen según una clave. Esa clave se denomina clave de particionamiento, clave de distribución o shard key, dependiendo del motor. La elección de esa clave es una de las decisiones más importantes en el diseño de bases distribuidas.

Por ejemplo, una plataforma de auditoría continua que registra eventos de usuarios podría particionar los datos por `id_usuario`, `empresa`, `fecha_evento`, `region`, `tipo_evento` o una combinación de varios campos. Cada decisión tiene consecuencias distintas.

Si se particiona por `fecha_evento`, todos los eventos de un mismo día podrían concentrarse en la misma partición. Esto puede ser útil para consultas históricas por rango temporal, pero peligroso si el día actual concentra la mayor cantidad de escrituras. Si se particiona por `id_usuario`, la distribución puede ser más equilibrada si

hay muchos usuarios activos. Pero si algunos usuarios generan millones de eventos y otros generan pocos, también puede aparecer desbalanceo. Si se particiona por región, puede facilitar la localización geográfica de datos, aunque puede generar concentración si una región domina el volumen.

En Cassandra, la clave de partición define dónde se almacenan los datos dentro del cluster y condiciona directamente el patrón de consulta. Cassandra no está pensada para consultar libremente cualquier columna como si fuera una base relacional tradicional; está diseñada para responder eficientemente consultas alineadas con su modelo de partición y clustering. Por eso, en Cassandra se suele diseñar la tabla en función de la consulta que se necesita resolver.

En MongoDB, el sharding permite distribuir datos entre múltiples shards. Para que funcione correctamente, la elección de la shard key debe favorecer la distribución de datos y consultas. Una mala shard key puede provocar que una parte del cluster trabaje mucho más que otra.

El particionamiento permite crecer en volumen y rendimiento, pero exige anticipar el comportamiento de la aplicación. No es una decisión puramente técnica; es una decisión de modelado orientada al acceso. La pregunta no es solamente “cómo divido los datos”, sino “cómo se van a escribir, leer, filtrar, agrupar y consultar esos datos”.

Replicación

La replicación consiste en mantener copias de los datos en más de un nodo. Mientras que el particionamiento divide datos, la replicación los copia. Ambas técnicas pueden combinarse: un sistema puede particionar los datos en shards y, además, replicar cada shard en varios nodos.

La finalidad principal de la replicación es aumentar la disponibilidad, la tolerancia a fallos y la capacidad de recuperación. Si un dato existe en un solo nodo y ese nodo falla, el dato queda temporalmente inaccesible. Si el dato está replicado en varios nodos, el sistema puede continuar operando con alguna de las copias disponibles.

Un concepto central es el factor de replicación. El factor de replicación indica cuántas copias de un dato se mantienen en el cluster. Si el factor de replicación es 3, cada dato se almacena en tres réplicas. Esto no significa necesariamente que haya tres bases completas idénticas; depende del modelo de distribución. En Cassandra, por ejemplo, los datos se distribuyen y se replican entre nodos según la estrategia de replicación y el factor definido. En MongoDB, un replica set mantiene copias del mismo conjunto de datos entre un nodo primario y nodos secundarios. En Redis, la replicación básica sigue un modelo líder-réplica, donde las réplicas intentan mantenerse como copias del nodo principal.

La replicación introduce un beneficio evidente: si una copia falla, puede usarse otra. Pero también introduce un problema: las copias deben mantenerse sincronizadas. Cuando una aplicación escribe un dato, el sistema debe decidir cuándo considera que

esa escritura fue exitosa. ¿Alcanza con que la reciba un nodo? ¿Deben confirmarla dos nodos? ¿Debe confirmarla la mayoría? ¿Debe llegar a todas las réplicas?

Esta decisión afecta la durabilidad, la latencia y la consistencia. Si se exige confirmación de muchas réplicas, aumenta la seguridad de que el dato no se pierda, pero también aumenta el tiempo de respuesta. Si se exige confirmación de una sola réplica, la respuesta es más rápida, pero puede haber mayor riesgo ante una falla inmediata.

MongoDB trabaja este problema mediante el concepto de write concern. El write concern define cuántos miembros que almacenan datos deben reconocer una escritura antes de considerarla exitosa. Un write concern de mayoría implica que el nodo primario espera confirmación de la mayoría de los miembros correspondientes antes de responder al cliente, lo que mejora la durabilidad, pero puede aumentar la latencia.

Cassandra utiliza niveles de consistencia configurables por operación. Un nivel de consistencia define cuántos nodos deben responder o confirmar una lectura o escritura antes de que la operación se considere exitosa. Esto permite ajustar el equilibrio entre disponibilidad, rendimiento y precisión de los datos según el caso de uso.

Redis, por su parte, suele utilizarse para datos de acceso muy rápido, muchas veces en memoria. La replicación permite disponer de copias, pero el nivel de persistencia, recuperación y consistencia depende de la arquitectura seleccionada y de la configuración. Redis también ofrece persistencia en disco mediante distintas alternativas, pero su uso típico en arquitecturas modernas suele estar asociado a caché, sesiones, estados transitorios, contadores y estructuras de baja latencia.

Alta disponibilidad y recuperación

La alta disponibilidad, o High Availability, es la capacidad de un sistema para seguir prestando servicio aunque existan fallos parciales. No significa que nunca falle nada. Significa que el sistema está diseñado para detectar fallos, aislarlos, recuperarse y continuar operando con el menor impacto posible.

En una arquitectura de base de datos, la alta disponibilidad puede apoyarse en replicación, failover automático, monitoreo, balanceo de carga, particionamiento, backups, logs de recuperación y mecanismos de elección de nuevos nodos principales.

El failover es el proceso por el cual el sistema transfiere automáticamente el servicio desde un nodo fallido hacia otro nodo disponible. Por ejemplo, en un replica set de MongoDB, si el nodo primario deja de estar disponible, los miembros restantes pueden realizar una elección para seleccionar un nuevo primario. En Redis, Sentinel proporciona alta disponibilidad para configuraciones no clusterizadas, realizando monitoreo, notificaciones y actuando como proveedor de configuración para clientes.

La recuperación de datos no debe confundirse con la disponibilidad. Un sistema puede estar disponible y responder, pero haber perdido información reciente. También puede preservar estrictamente la información, pero dejar de responder temporalmente

mientras se recupera. Por eso, las decisiones de arquitectura deben equilibrar continuidad operativa y seguridad de los datos.

Un caso típico se observa en aplicaciones de comercio electrónico. Si un sistema de recomendaciones deja de responder, la plataforma puede seguir vendiendo. Si el contador de visualizaciones tiene unos segundos de atraso, el impacto puede ser bajo. Pero si el sistema de pagos confirma incorrectamente una operación o pierde una transacción aprobada, el impacto es crítico. No todos los datos tienen el mismo nivel de sensibilidad ni exigen el mismo nivel de consistencia.

Consistencia

La consistencia es uno de los conceptos más importantes y, al mismo tiempo, más confundidos en sistemas distribuidos. En términos generales, la consistencia se refiere a la coherencia de los datos entre distintas copias o nodos. Si un dato se modifica, la pregunta es: ¿todos los nodos ven inmediatamente el mismo valor?

En una consistencia fuerte, después de una escritura confirmada, cualquier lectura posterior debería devolver el valor actualizado o rechazar la operación si no puede garantizarlo. Esto reduce el riesgo de leer datos viejos, pero puede afectar la disponibilidad o la latencia si el sistema debe coordinar varias réplicas antes de responder.

En una consistencia eventual, el sistema acepta que durante un período breve puedan existir diferencias entre réplicas. Con el tiempo, si no se producen nuevas actualizaciones conflictivas, todas las réplicas convergerán hacia el mismo valor. Este modelo puede ser adecuado cuando la disponibilidad y el rendimiento son más importantes que la sincronización inmediata.

La consistencia eventual no significa desorden ni falta de control. Significa que el sistema admite una ventana temporal de divergencia. Por ejemplo, un contador de “me gusta”, una métrica de visualizaciones, un ranking temporal, un historial de navegación o una recomendación personalizada pueden tolerar que distintos nodos vean valores levemente diferentes durante algunos segundos. En cambio, una operación financiera, una reserva única de asiento o una aprobación crítica de auditoría suelen requerir mayor consistencia.

La clave está en comprender que la consistencia no es una propiedad abstracta, sino una decisión de negocio. Un sistema de salud no puede mostrar a un médico una medicación desactualizada si acaba de registrarse una alergia crítica. Un sistema bancario no puede permitir que dos retiros simultáneos gasten el mismo saldo. Pero una aplicación de noticias puede tolerar que el número de lecturas de una nota se actualice con algunos segundos de demora.

Disponibilidad

La disponibilidad se refiere a la capacidad del sistema para responder a las solicitudes. En el marco del teorema CAP, disponibilidad significa que toda solicitud recibida por un

nodo no fallido debe obtener una respuesta, aunque esa respuesta no necesariamente contenga el dato más reciente. Esta definición es más específica que el uso cotidiano de “alta disponibilidad” en infraestructura. En CAP, disponibilidad se relaciona con la decisión de seguir respondiendo aun cuando no se pueda garantizar una visión perfectamente consistente de los datos.

Un sistema altamente disponible intenta minimizar interrupciones. Para lograrlo, puede responder desde réplicas, redirigir tráfico, aceptar escrituras en varios nodos o continuar operando aunque una parte del cluster esté desconectada.

La disponibilidad tiene valor directo para el negocio. En una aplicación de delivery, cada minuto fuera de servicio implica pedidos perdidos. En una plataforma de streaming, una interrupción afecta la experiencia del usuario. En una billetera digital, la indisponibilidad puede impedir pagos. En una plataforma de auditoría continua, la indisponibilidad puede dejar de registrar eventos críticos.

Sin embargo, la disponibilidad no puede analizarse aislada. Responder siempre no es suficiente si la respuesta puede ser incorrecta para un proceso crítico. En algunos contextos es preferible rechazar una operación antes que aceptar una escritura que luego genere inconsistencias graves.

Tolerancia a particiones

La tolerancia a particiones es la capacidad de un sistema distribuido para continuar funcionando aunque existan fallas de comunicación entre nodos. Una partición de red ocurre cuando dos o más partes del sistema no pueden comunicarse entre sí, aunque los nodos sigan encendidos y funcionando.

Una partición no es necesariamente una caída total. Puede deberse a fallas de red, latencia extrema, pérdida de paquetes, problemas de conectividad entre regiones, interrupciones de un proveedor cloud, reglas de firewall mal configuradas o degradación de enlaces. Desde el punto de vista de los nodos, el problema es que no pueden saber con certeza si el otro nodo está caído, lento o simplemente incomunicado.

Este problema es central en sistemas distribuidos porque la red nunca es completamente confiable. En arquitecturas modernas, donde los sistemas pueden estar distribuidos entre centros de datos, zonas de disponibilidad o regiones geográficas, la posibilidad de fallas de comunicación debe asumirse como parte del diseño.

Cuando ocurre una partición, el sistema enfrenta una decisión crítica. Si dos partes del cluster no pueden comunicarse y ambas reciben operaciones, ¿deben seguir respondiendo? Si siguen respondiendo, podrían aceptar datos que luego entren en conflicto. Si dejan de responder, preservan consistencia, pero sacrifican disponibilidad.

Teorema CAP

El teorema CAP, asociado a Eric Brewer y formalizado por Seth Gilbert y Nancy Lynch, plantea que un sistema distribuido de datos no puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones en presencia de una partición de red. CAP corresponde a Consistency, Availability y Partition Tolerance: consistencia, disponibilidad y tolerancia a particiones. La prueba formal de Gilbert y Lynch consolidó la idea de que, ante ciertas condiciones de falla de red, se debe elegir entre mantener consistencia o mantener disponibilidad.

La formulación clásica suele resumirse como “solo se pueden elegir dos de tres”. Esa frase es útil como introducción, pero puede ser engañosa si se interpreta de manera rígida. En sistemas distribuidos reales, la tolerancia a particiones no es una opción decorativa: las particiones pueden ocurrir. Por eso, cuando ocurre una partición, la decisión efectiva suele ser entre priorizar consistencia o priorizar disponibilidad. Brewer, al revisar CAP años después, señaló que la idea de “dos de tres” simplifica demasiado el problema, porque los sistemas reales pueden manejar particiones explícitamente y optimizar distintos grados de consistencia y disponibilidad según el contexto.

La consistencia en CAP no debe confundirse con la “C” de ACID. ACID significa Atomicity, Consistency, Isolation, Durability: atomicidad, consistencia, aislamiento y durabilidad. En ACID, consistencia se relaciona con preservar reglas e invariantes de la base de datos dentro de una transacción. En CAP, consistencia se refiere a que las lecturas observen el valor más reciente o una visión equivalente entre nodos. Son conceptos relacionados con la corrección de datos, pero pertenecen a marcos teóricos distintos.

El teorema CAP no dice que un sistema sea siempre consistente o siempre disponible. Plantea el conflicto en situación de partición. En condiciones normales, un sistema puede ofrecer buena consistencia, buena disponibilidad y tolerancia a fallos. El dilema aparece cuando la red se rompe y el sistema debe decidir cómo comportarse.

Arquitecturas CP y AP

Una arquitectura CP prioriza Consistency y Partition Tolerance: consistencia y tolerancia a particiones. Ante una partición, el sistema puede rechazar operaciones, bloquear escrituras o dejar de responder en una parte del cluster si no puede garantizar consistencia. El objetivo es evitar que existan versiones conflictivas del dato.

Una arquitectura AP prioriza Availability y Partition Tolerance: disponibilidad y tolerancia a particiones. Ante una partición, el sistema sigue respondiendo aunque pueda aceptar o devolver datos temporalmente desactualizados. Luego necesitará mecanismos de reconciliación, sincronización o convergencia.

La clasificación CP o AP no debe usarse como etiqueta absoluta. Muchos motores permiten configurar comportamientos intermedios. Cassandra, por ejemplo, se asocia con disponibilidad y tolerancia a particiones, pero permite niveles de consistencia ajustables por operación. MongoDB suele ubicarse más cerca de CP en configuraciones

típicas de replica set, pero también ofrece distintas opciones de read concern y write concern. Redis depende fuertemente de la arquitectura: un Redis simple, Redis con réplicas, Redis Sentinel y Redis Cluster tienen comportamientos y objetivos distintos.

Por eso, CAP debe usarse como herramienta de razonamiento, no como clasificación mecánica. La pregunta correcta no es “qué letra tiene esta base”, sino “qué sacrifica esta configuración cuando hay una partición y qué impacto tiene eso en el negocio”.

MongoDB: documentos, replicación y sharding

MongoDB es una base de datos documental. Almacena información en documentos flexibles similares a JSON, internamente en BSON, que significa Binary JSON. BSON permite representar estructuras de datos ricas con objetos embebidos, arrays, fechas, números y otros tipos. Esta flexibilidad hace que MongoDB sea adecuado para perfiles de usuario, catálogos de productos, documentos de auditoría, contenido semiestructurado, configuraciones, órdenes, registros operativos y entidades que no siempre tienen la misma estructura.

En términos de escalabilidad y disponibilidad, MongoDB utiliza dos conceptos centrales: replica sets y sharding.

Un replica set es un grupo de instancias mongod que mantienen el mismo conjunto de datos. Normalmente existe un nodo primario que recibe escrituras y nodos secundarios que replican los cambios. Si el primario falla, el replica set puede elegir un nuevo primario. Este modelo favorece la consistencia de las escrituras porque evita que múltiples nodos primarios acepten escrituras independientes sobre el mismo conjunto de datos.

El sharding en MongoDB permite distribuir datos entre múltiples shards. Cada shard contiene una parte del conjunto total. Esto permite escalar horizontalmente cuando el volumen de datos o la carga de operaciones exceden la capacidad de un único replica set. La elección de la shard key es crítica, porque define cómo se distribuyen los documentos.

MongoDB también permite configurar write concern y read concern. Write concern controla cuántos miembros deben confirmar una escritura antes de considerarla exitosa. Read concern controla la consistencia y el aislamiento de los datos leídos desde replica sets y clusters shardeados. Estas configuraciones permiten ajustar el equilibrio entre rendimiento, durabilidad y consistencia.

Desde el punto de vista de CAP, MongoDB suele explicarse como una tecnología que, en configuraciones típicas de replica set, prioriza consistencia y tolerancia a particiones. Si el primario no está disponible o no se puede alcanzar una mayoría adecuada, puede haber interrupciones temporales hasta que el sistema elija un nuevo primario o recupere condiciones seguras. Esto implica que, ante ciertos fallos, MongoDB puede preferir no aceptar escrituras antes que aceptar escrituras conflictivas.

Un caso actual y habitual para MongoDB es una plataforma de e-commerce con catálogo flexible. Cada categoría de producto puede tener atributos diferentes: un celular tiene memoria, sistema operativo y tamaño de pantalla; una zapatilla tiene talla, color y material; un perfume tiene volumen, familia olfativa e intensidad. Modelar todos esos atributos en tablas relacionales rígidas puede ser complejo. MongoDB permite representar cada producto como un documento con estructura variable. Si además la plataforma crece, el catálogo puede distribuirse mediante sharding y protegerse mediante replica sets.

Otro caso es una plataforma de auditoría que registra eventos con detalles variables. Un evento de modificación de proveedor puede tener campos distintos de un evento de aprobación de pago o de un evento de cambio de permisos. En lugar de forzar todos los eventos a una misma estructura tabular, puede utilizarse un modelo documental donde los campos comunes conviven con detalles específicos embebidos.

Cassandra: distribución masiva y consistencia configurable

Apache Cassandra es una base de datos distribuida orientada a grandes volúmenes de datos y alta disponibilidad. Su modelo suele clasificarse dentro de las bases wide-column o de columnas amplias. Aunque utiliza tablas, filas y columnas, no debe confundirse con un modelo relacional clásico. Cassandra está diseñada para distribuir datos entre múltiples nodos, soportar escrituras intensivas y operar sin un único punto central de falla.

Una característica clave de Cassandra es que todos los nodos pueden cumplir roles equivalentes dentro del cluster. No depende de un único nodo primario para todas las escrituras. Los datos se distribuyen mediante una clave de partición y se replican según el factor de replicación definido. El diseño de tablas en Cassandra se realiza a partir de las consultas que se quieren responder. Esto significa que no se modela primero una estructura normalizada y luego se consultan libremente los datos; se diseñan tablas orientadas a patrones concretos de acceso.

Cassandra permite configurar niveles de consistencia por operación. Un nivel de consistencia determina cuántas réplicas deben responder o confirmar una operación antes de considerarla exitosa. Por ejemplo, con un factor de replicación de 3, una escritura podría requerir confirmación de una réplica, de dos, de la mayoría o de todas, según la configuración. Esta capacidad se conoce como consistencia ajustable o tunable consistency.

El modelo de Cassandra se asocia frecuentemente con disponibilidad y tolerancia a particiones. Esto no significa que ignore la consistencia, sino que permite ajustar el nivel de consistencia requerido según la operación. Para algunos datos puede priorizar disponibilidad; para otros, puede exigir mayor confirmación de réplicas.

Un ejemplo claro es una plataforma de IoT, Internet of Things o Internet de las Cosas, que recibe mediciones de sensores industriales cada segundo. Si miles de dispositivos envían temperatura, presión, vibración o ubicación, la base debe absorber escrituras

constantes. Cassandra es adecuada para este tipo de cargas porque puede distribuir las escrituras entre nodos y escalar agregando capacidad horizontal.

Otro caso es una plataforma de eventos digitales que registra clics, vistas, búsquedas, reproducciones, logs de seguridad o actividad de usuarios. Estos datos suelen tener enorme volumen y se consultan por usuario, fecha, dispositivo o tipo de evento. Cassandra puede almacenar series de eventos distribuidas y replicadas, siempre que el modelo de consulta haya sido diseñado correctamente.

La advertencia conceptual es importante: Cassandra no es conveniente para cualquier problema. Si se necesitan consultas ad hoc complejas, joins frecuentes, transacciones relacionales o exploración flexible de datos sin patrón definido, Cassandra puede no ser la mejor opción. Su potencia aparece cuando el patrón de consulta es claro, el volumen es alto, la escritura es intensa y la disponibilidad distribuida es prioritaria.

Redis: velocidad, memoria y datos de baja latencia

Redis es una base de datos clave-valor en memoria. Su nombre proviene de Remote Dictionary Server. Aunque suele presentarse como clave-valor, Redis soporta estructuras de datos más ricas: strings, hashes, lists, sets, sorted sets, streams, bitmaps y otras. Su principal fortaleza es la velocidad, porque trabaja principalmente en memoria RAM.

Redis se utiliza habitualmente para caché, sesiones, contadores, rankings, colas simples, estados temporales, control de intentos de login, bloqueos distribuidos, carritos temporales, tokens, TTL y datos que requieren acceso de baja latencia. TTL significa Time To Live, tiempo de vida. Un dato con TTL expira automáticamente después de un período determinado.

Por ejemplo, un sistema de autenticación puede registrar intentos fallidos con una clave como login:intentos:usuario_303. Si el usuario supera cierta cantidad de intentos, puede crearse una clave login:bloqueado:usuario_303 con expiración de diez minutos. Redis es adecuado para este caso porque permite incrementar contadores rápidamente y manejar expiraciones automáticas.

Redis también permite replicación líder-réplica, donde una o más réplicas intentan mantenerse como copias del nodo principal. Redis Sentinel agrega capacidades de alta disponibilidad para configuraciones no clusterizadas, incluyendo monitoreo, notificaciones y failover. Redis Cluster permite distribuir datos entre múltiples nodos usando particionamiento.

Desde CAP, Redis no debe clasificarse de manera única sin mirar la arquitectura. Un Redis simple no representa por sí mismo un sistema distribuido complejo. Redis con réplicas, Redis Sentinel o Redis Cluster sí introduce decisiones sobre disponibilidad, replicación, failover y particionamiento. En muchos usos, Redis prioriza rendimiento y disponibilidad operativa para datos rápidos o transitorios. Cuando se requiere durabilidad estricta, se deben configurar adecuadamente sus mecanismos de persistencia o utilizarlo como complemento de otra base de datos principal.

Un caso típico es una plataforma bancaria o fintech que utiliza una base transaccional para saldos y movimientos definitivos, pero Redis para controlar sesiones, tokens, límites temporales, intentos fallidos, caché de datos no críticos y reglas de velocidad. Redis no reemplaza necesariamente a la base principal; la complementa como capa rápida.

Otro caso es una plataforma de streaming que necesita mostrar rankings, recomendaciones recientes o contadores de reproducciones. Algunos datos pueden persistirse luego en un sistema analítico, mientras Redis responde en tiempo real a las operaciones de baja latencia.

Administración e interfaces de administración

Administrar una base NoSQL distribuida implica monitorear nodos, réplicas, particiones, uso de memoria, latencia, errores, espacio en disco, estado del cluster, tiempos de respuesta, logs, políticas de backup, seguridad y recuperación. La administración no se limita a crear bases y colecciones; incluye comprender cómo se comporta el sistema bajo carga y cómo responde ante fallos.

Las interfaces de administración pueden ser gráficas, de consola o servicios administrados en la nube. MongoDB puede administrarse mediante MongoDB Shell, Compass, Atlas y herramientas de monitoreo. Cassandra utiliza CQL, Cassandra Query Language, junto con herramientas operativas como nodetool para inspeccionar estado del cluster, nodos, compactaciones, reparación y otros aspectos. Redis se administra mediante Redis CLI, herramientas de monitoreo, Sentinel, Cluster y plataformas administradas.

CQL significa Cassandra Query Language. Se parece visualmente a SQL en algunas instrucciones, pero no funciona con la misma lógica relacional. No debe interpretarse como un SQL tradicional. Está orientado al modelo distribuido de Cassandra y sus restricciones de consulta.

CLI significa Command Line Interface, interfaz de línea de comandos. Es una forma de administrar sistemas mediante instrucciones escritas en consola. Aunque las interfaces gráficas facilitan la operación, las interfaces de línea de comandos siguen siendo fundamentales para diagnóstico, automatización y administración técnica.

Una buena administración de bases NoSQL requiere observar métricas operativas y no solamente resultados funcionales. Por ejemplo, una consulta puede devolver datos correctos, pero generar un escaneo ineficiente en muchos nodos. Una base puede responder rápido en pruebas pequeñas, pero degradarse cuando el volumen crece. Una arquitectura puede funcionar con un nodo, pero fallar al distribuirse si la clave de particionamiento está mal diseñada.

Casos aplicados de decisión arquitectónica

Una plataforma de comercio electrónico omnicanal maneja clientes, productos, inventario, carritos, pagos, navegación, recomendaciones y promociones. No todos esos datos deberían almacenarse necesariamente en la misma tecnología.

El catálogo de productos puede resolverse con MongoDB si las categorías tienen atributos variables. Los eventos de navegación pueden almacenarse en Cassandra si se generan grandes volúmenes de clics, búsquedas, vistas y comportamiento histórico. Los carritos temporales, sesiones activas, intentos de login y promociones de corta duración pueden gestionarse con Redis por su velocidad y capacidad de expiración automática. La información financiera definitiva podría mantenerse en una base transaccional con garantías fuertes.

En este caso, CAP no se aplica de manera uniforme a toda la empresa. Se aplica por componente. El carrito temporal puede priorizar disponibilidad y velocidad. El pago requiere consistencia fuerte. Las recomendaciones pueden tolerar consistencia eventual. El catálogo necesita disponibilidad, pero algunas actualizaciones pueden publicarse de manera controlada. Los eventos históricos pueden aceptar escritura distribuida y sincronización posterior.

Una plataforma de auditoría continua también puede combinar modelos. Los eventos de actividad pueden almacenarse en Cassandra si el volumen es muy alto y las consultas se orientan por usuario, fecha o proceso. Los documentos de evidencia, hallazgos o configuraciones de controles pueden almacenarse en MongoDB por su flexibilidad documental. Redis puede utilizarse para reglas rápidas, como detectar si un usuario superó cierto umbral de acciones en pocos minutos o si una operación ya fue marcada como sospechosa en una ventana temporal.

En un sistema de salud, las decisiones cambian. La historia clínica, alergias, medicación activa y diagnósticos críticos requieren consistencia fuerte y trazabilidad. Algunas métricas secundarias, como estadísticas agregadas de uso o paneles de actividad, pueden tolerar demoras. Esto demuestra que no existe una única respuesta tecnológica. El tipo de dato y el riesgo asociado determinan el diseño.

Errores frecuentes de interpretación

Un error común es pensar que replicar y particionar significan lo mismo. Particionar divide los datos; replicar los copia. Un sistema puede hacer una cosa, la otra o ambas.

Otro error es creer que más nodos siempre implican mejor rendimiento. Si la clave de particionamiento es mala, si las consultas no están alineadas con el modelo o si la red introduce demasiada latencia, agregar nodos puede no resolver el problema.

También es frecuente interpretar CAP como una regla fija para etiquetar bases de datos: “MongoDB es CP”, “Cassandra es AP”, “Redis es AP”. Esa simplificación puede servir como primera aproximación, pero es insuficiente. Las configuraciones concretas, los niveles de consistencia, la topología, la replicación, las políticas de lectura y escritura, y el comportamiento ante fallos son los que determinan la arquitectura real.

Otro error es pensar que consistencia eventual significa datos incorrectos. En realidad, significa que pueden existir diferencias temporales entre réplicas hasta que el sistema converge. Puede ser perfectamente aceptable para ciertos usos y completamente inaceptable para otros.

También es incorrecto asumir que NoSQL siempre reemplaza al modelo relacional. En arquitecturas modernas, lo habitual es la coexistencia. Una organización puede usar una base relacional para transacciones críticas, MongoDB para documentos flexibles, Cassandra para eventos masivos, Redis para baja latencia y un data warehouse o lakehouse para analítica consolidada.

Cierre conceptual

La escalabilidad en NoSQL no se reduce a almacenar más datos. Implica decidir cómo se distribuyen, replican, consultan, protegen y recuperan esos datos. La replicación mejora disponibilidad y recuperación, pero obliga a gestionar consistencia. El particionamiento permite distribuir volumen y carga, pero exige elegir claves adecuadas. La disponibilidad permite continuidad operativa, pero puede entrar en tensión con la consistencia. La tolerancia a particiones recuerda que en sistemas distribuidos la red puede fallar y la arquitectura debe estar preparada.

El teorema CAP aporta un marco conceptual para comprender esas tensiones. No debe usarse como una fórmula rígida, sino como una herramienta de diseño. Cada sistema debe decidir qué datos requieren consistencia fuerte, qué datos pueden aceptar consistencia eventual, qué componentes deben seguir funcionando ante fallos y qué riesgos son tolerables.

MongoDB, Cassandra y Redis representan tres formas distintas de abordar problemas NoSQL. MongoDB ofrece flexibilidad documental con mecanismos de replicación y distribución. Cassandra ofrece escalabilidad horizontal, escritura distribuida y consistencia configurable. Redis ofrece velocidad extrema para datos en memoria, estados temporales y operaciones de baja latencia. La arquitectura correcta no surge de elegir la tecnología más conocida, sino de comprender el problema, el patrón de acceso, el volumen, la criticidad del dato, la tolerancia al error y la necesidad real de disponibilidad.